

Chapter 6 Summary

Full-stack Development with Node.js

Sections 6.1 – 6.5

6.1 Full-stack Development (Node)

Front-end vs. Back-end

Web applications rely on two sides that work together: the front-end runs in the browser, and the back-end runs on the server.

Client-side (front-end) — Runs in the browser using HTML, CSS, and JavaScript.

Server-side (back-end) — Runs on the web server using PHP, Python, Node.js, ASP.NET, etc., along with databases.

Full-stack developer — A developer proficient in client, server, data modeling, and UI — but usually specialized in a few areas.

Primary Layers of the Full Stack

Layer	Description
User Interface (UI)	The visual part of the application the user interacts with.
Business Logic	Programming logic that decides how data is created, displayed, stored, and changed.
Data Modeling	Representing and storing data in relational or non-relational databases.
API	Programmable actions on the underlying data; used by the front-end to talk to the back-end.
Server & Hosting	Network throughput, cloud storage, virtualization, hardware, data redundancy.
Testing Framework	Automated tests verifying components work correctly, alone and together.

Web Hosting

Large companies host their own servers; smaller ones usually rent from a web hosting company. Four main factors when choosing a host:

- **Reliability** — uptime guarantees, backup frequency.
- **Flexibility** — ability to scale up (e.g., a VPS or cloud like AWS).

- **Security** — encryption, dedicated servers, protection from malware.
- **Price** — free (ad-supported) up to expensive dedicated hosting.

Virtual Private Server (VPS) — An autonomous server hosted on a physical machine alongside other virtual servers.

Malware — Malicious software designed to damage a system or perform unwanted actions.

Linux servers (open-source: Apache + PHP/Python/Ruby + MySQL) are usually cheaper than **Windows servers** (IIS + ASP.NET + SQL Server).

Server-side Programming Languages

Language / Platform	Description
PHP	Scripting language by Rasmus Lerdorf (1994). Most popular server-side language.
ASP.NET	Microsoft platform (2002), uses C# or VB.NET.
Python	General-purpose language with frameworks Django, Flask, web2py.
Ruby on Rails	Web framework written in Ruby, created by David Heinemeier Hansson (2004).
Java	Used for applets (front-end) and servlets, JSP, and web APIs (back-end).
Node.js	JavaScript runtime environment, created by Ryan Dahl (2009).

Dynamic Webpages, SPAs, and Web APIs

Dynamic webpage — A webpage generated on the server when requested, often personalized to the user.

Single Page Application (SPA) — A web app providing a desktop-like experience in a single page (e.g., Gmail, Google Docs). All resources load once; later interactions fetch small pieces dynamically.

Web API — A collection of functions invoked using HTTP. SPAs use JavaScript Ajax calls to talk to web APIs.

Databases

Relational databases store data in tables and use **SQL** (Structured Query Language). Examples: MySQL, PostgreSQL, Oracle, SQL Server.

Non-relational (NoSQL) databases store data in flexible formats:

- **Document database** — JSON-format documents with nesting. Ex: MongoDB.
- **Key-value database** — Values associated with unique keys. Ex: Redis.
- **Object database** — Objects from OO programming languages. Ex: Caché.
- **Column database** — Large data with pointers distributed across columns. Ex: HBase.

- **Graph database** — Stores graph structures (nodes and edges). Ex: Neo4j.

Client-side Technologies

- **HTML preprocessor** — Converts markup into HTML. Ex: Haml, Markdown, Slim, Pug.
- **CSS preprocessor** — Converts CSS-like languages into CSS. Ex: Sass, Less, Stylus.
- **UI library** — Creates widgets like sliders, dropdowns. Ex: jQuery UI, Bootstrap, React, Vue.js.
- **CSS front-end framework** — For responsive design. Ex: Bootstrap, Foundation, Skeleton.
- **Compile-to-JavaScript language** — Languages compiled to JS. Ex: TypeScript, CoffeeScript, Haxe.
- **JavaScript framework** — Organizes JS code and UI/data flow. Ex: AngularJS, Ember, Backbone.

Testing

Developers must test the full technology stack. Frameworks like Selenium automate UI testing. Common test types:

- **Functionality testing** — verifies each function works as expected.
- **Interface testing** — tests front-end ↔ back-end ↔ database interaction.
- **Usability testing** — checks ease of use for the intended purpose.
- **Compatibility testing** — tests across browsers, OSes, and platforms.
- **Performance testing** — verifies behavior under load.
- **Security testing** — ensures privacy and data integrity.

6.2 Getting Started with Node.js

Introduction to Node.js

Node.js — A JavaScript runtime environment primarily used to run server-side web applications.

Key benefits:

- Event-driven, non-blocking I/O architecture handles high loads efficiently.
- JavaScript runs on both client and server, simplifying development.
- Simple module system for code sharing and reuse.
- Works seamlessly with MongoDB (JSON throughout the stack).

Used by Netflix, Walmart, eBay, LinkedIn. npm has more publicly available modules than any other language ecosystem.

Installing and Running Node.js

Install Node.js from the official website. After installing:

- **Interactive shell** — type `node` in the terminal to start it; `.exit` quits.
- **Run a script** — `node myprogram.js` executes a `.js` file.
- **Output to console** — use `console.log("...")`.

Creating a Simple Web Server

The `http` module lets Node.js create a basic web server using `createServer()` and `listen()`. The `require()` function imports CommonJS modules.

```
const http = require("http");

http.createServer(function(request, response) {
  response.writeHead(200, {"Content-Type": "text/html"});
  response.write("<h1>Hello, Node.js!</h1>");
  response.end();
}).listen(3000);
```

- **writeHead()** — sets the status code and response headers.
- **write()** — sends content to the response body.
- **end()** — sends the response to the browser.
- **listen(port)** — starts the server on a port; Ctrl+C kills it.

Projects and npm

Node.js project — A directory containing JS files, packages, and config files.

Package — A directory with one or more modules plus a package.json file.

npm (Node Package Manager) — Installs, updates, and removes packages. Installed with Node.js.

npm has two install modes:

- **Local mode** — installs into the project's node_modules/ folder. Ideal for project dependencies.
- **Global mode (-g)** — installs into {prefix}/node_modules. Used for command-line tools (e.g., nodemon).

Common npm Commands

Command	Description	Example
config	Manage npm configuration	npm config get prefix
install	Install a package (use -g for global)	npm install nodemon -g
list	List installed packages	npm list
update	Update a package	npm update lodash
uninstall	Remove a package	npm uninstall lodash

package.json and package-lock.json

package.json lists the project's name, version, license, scripts, and dependencies. Create it manually or with `npm init`. When present, run `npm install` to install all dependencies at once.

package-lock.json locks specific dependency versions so the same versions install on every machine.

Semantic Versioning (major.minor.patch)

- **major** — breaking changes / new functionality (e.g., 2.0.0).
- **minor** — bug fixes and minor changes (e.g., 1.2.0).
- **patch** — small bug fix on a minor version (e.g., 1.2.3).

The caret (^) in `^1.9.1` means the highest version where the major number stays 1.

6.3 Modules

Why Modules?

Module — A JavaScript file containing variables, functions, or classes that can be exported and reused.

Modules avoid naming conflicts (e.g., two files both declaring a `max` variable) and make code maintainable and reusable.

Two common module formats:

- **ES Modules (ECMAScript)** — supported by all major browsers.
- **CommonJS Modules** — commonly used in Node.js.

Module Exports

Multiple items (use braces):

```
function sayHello() { alert("Hello!"); }  
function sayHi() { alert("Hi!"); }  
export { sayHello, sayHi };
```

Default export (only one per module, no braces):

```
function sayGoodbye() { alert("Goodbye!"); }  
export default sayGoodbye;
```

Module Imports

Named imports use braces; default imports don't:

```
import { sayHello, sayHi } from "./hello.js";  
import sayGoodbye from "./goodbye.js";
```

Important rules:

- In HTML, scripts must use `<script type="module">` for import to work.
- The `./` prefix indicates the file is in the same directory.
- Modules must be served from a web server (the `file://` protocol won't load them).
- Some developers use a `.mjs` extension to distinguish modules from regular JS files.

6.4 Express

Express Server

Express — A popular web application framework for Node.js that lets you build web servers with less code.

Install with `npm install express`. A minimal Express server:

```
const express = require("express");
const app = express();

// Serve static files from the public dir
app.use(express.static("public"));

app.listen(3000, function() {
  console.log("Listening on port 3000...");
});
```

`express.static("public")` serves any file from the `public/` directory (HTML, images, etc.).

Routes

Route — A specific URL path + HTTP method with an assigned callback function.

Format: `app.method(path, callback)`

- **app** — the Express instance.
- **method** — HTTP method (get, post, put, delete, etc.).
- **path** — the URL path.
- **callback** — function executed when the route matches; gets (req, res).

```
app.get("/hello", function(req, res) {
  res.send("<h1>Hello, Express!</h1>");
});

app.post("/goodbye", function(req, res) {
  res.send("<h1>Goodbye, Express!</h1>");
});
```

Middleware

Middleware function — A function that examines or modifies the request/response objects before the route runs.

Middleware takes three parameters:

- **req** — the HTTP request object.
- **res** — the HTTP response object.
- **next** — callback that lets the next middleware (or the route) run.

Enable with `app.use(middlewareFunction)`. Middleware must be registered BEFORE the routes it should run for.

```
const logRequest = function(req, res, next) {  
  console.log(`Request: ${req.method} for ${req.path}`);  
  next(); // pass control to next middleware/route  
};  
  
app.use(logRequest);
```

Third-Party Middleware

Middleware	Purpose
morgan	Logs HTTP request information.
cookie-parser	Parses the cookie header in an HTTP request.
errorhandler	Aids debugging during development.
csurf	Protects against Cross-Site Request Forgery (CSRF).
compression	Compresses response bodies.
express-session	Manages session data on the server.

Example: `npm install morgan`, then `app.use(morgan("dev"))`; logs every request in the console.

6.5 Express Request Data

Query String Parameters

Express automatically parses the part of a URL after the `?` and stores it in `req.query`.

```
// URL: http://localhost:3000/hello?name=Bob&age=21

app.get("/hello", function(req, res) {
  const html =
    `<h1>Hello, ${req.query.name}!</h1>
    <p>You are ${req.query.age} years old.</p>`;
  res.send(html);
});
```

If a parameter is missing, `req.query.someName` is undefined.

Posting Form Data

When a form posts data, it is URL-encoded in the request body. The `express.urlencoded()` middleware parses that body and attaches the values to `req.body`.

```
app.use(express.urlencoded({ extended: false }));

app.post("/hello", function(req, res) {
  const html = `<h1>Hello, ${req.body.name}!</h1>
    <p>You are ${req.body.age} years old.</p>`;
  res.send(html);
});
```

Content-Type for form posts is `application/x-www-form-urlencoded`.

Route Parameters

Route parameter — A string in the URL path (defined with a colon, e.g., `:username`) that captures a dynamic value.

Captured values appear in `req.params`.

```
// URL: http://localhost:3000/users/jblack

app.get("/users/:username", function(req, res) {
  const username = req.params.username; // "jblack"
  res.send("<h1>Profile for " + username + "</h1>");
});
```

```
});
```

Route Parameter Middleware

The `app.param()` method defines middleware that runs BEFORE any route using that parameter. It has 4 args: (`req`, `res`, `next`, `value`).

```
app.param("username", function(req, res, next, username) {
  if (username === "jblack") {
    req.user = { name: "Jack Black", username };
  } else {
    req.user = { name: "Unknown", username };
  }
  next(); // MUST be called or request hangs
});

app.get("/users/:username", function(req, res) {
  res.send("<h1>Profile for " + req.user.name + "</h1>");
});
```

Useful methods on `res`:

- **`res.send(html)`** — sends an HTML response.
- **`res.json(obj)`** — encodes an object as JSON; Content-Type becomes `application/json`.
- **`res.status(code)`** — sets the response status code (e.g., 404 for not found).

★ Important from the Slides

Quick-reference of the most important concepts to remember for the exam.

Core Definitions to Memorize

- **Front-end** — client-side; runs in browser (HTML, CSS, JS).
- **Back-end** — server-side; runs on server (Node.js, PHP, etc.) + databases.
- **Full-stack developer** — knows both sides, but usually specializes.
- **SPA** — Single Page Application — loads once, updates dynamically via Ajax to web APIs.
- **Web API** — collection of functions invoked over HTTP.
- **VPS** — Virtual Private Server — autonomous server sharing a physical machine.
- **Module** — JS file exporting reusable code; avoids naming conflicts.
- **Middleware** — function that runs between the request and the route, with (req, res, next).
- **Route** — URL path + HTTP method + callback.

Node.js Essentials

- Node.js runs on the SERVER, NOT in the browser.
- `require()` imports a CommonJS module.
- `node filename.js` runs a Node program; `node` alone opens the interactive shell.
- Ctrl+C kills a running server.
- Local install: `npm install pkg` → goes in project's `node_modules/`.
- Global install: `npm install pkg -g` → goes in `{prefix}/node_modules`.
- Semantic versioning: `major.minor.patch` — the caret (^) lets minor+patch update but locks major.

Express — Must Remember

- `express.static("public")` serves files from `public/` automatically.
- Route format: `app.METHOD(path, callback)`.
- Method must match exactly: a POST to a GET route does NOT match.
- **Middleware runs in ORDER.** If `app.use()` comes after `app.get()`, the middleware won't run for that route.
- Middleware MUST call `next()` — otherwise the request hangs forever.

Three Ways to Pass Data to Express

Source	Where it lives	Example
Query string	<code>req.query</code>	<code>/hello?name=Bob</code> → <code>req.query.name</code>

Source	Where it lives	Example
Form body (POST)	req.body (needs express.urlencoded)	name=Bob → req.body.name
Route parameter	req.params	/users/:id → req.params.id

Useful Response Methods

- **res.send(html)** — send HTML or text.
- **res.json(obj)** — send object as JSON; sets Content-Type to application/json.
- **res.status(code)** — set status code (e.g., 404, 401, 500). Chainable: res.status(404).json({...}).

Common Pitfalls (Likely Exam Traps)

- Forgetting `next ( )` in middleware → request never reaches the route.
- Forgetting `express.urlencoded ( )` → req.body is undefined.
- Forgetting `type="module"` on a script tag → import statements fail.
- URL path is case-sensitive: `/Droids` ≠ `/droids`.
- Using HTTP method that doesn't match the route — POST to a GET-only route returns 404.
- Importing without `./` for local files → path resolution error.

Quick Recall Facts

- Linux servers are usually cheaper than Windows servers (open-source).
- PHP is the most popular server-side language.
- npm has more public modules than any other language ecosystem.
- MongoDB is a DOCUMENT database (NoSQL), stores JSON.
- SQL is for RELATIONAL databases.
- Default Node.js port for examples is 3000; web servers typically use 80 (HTTP) or 443 (HTTPS).
- Status codes: 200 OK, 201 Created, 401 Unauthorized, 404 Not Found, 500 Server Error.